```
    begin
➤     class_eval(method_definition, __FILE__, __LINE__)
    rescue SyntaxError => err
      generated_methods.delete(attr_name)
      if logger
        logger.warn "Exception occurred during reader method compilation."
        logger.warn "Maybe #{attr_name} is not a valid Ruby identifier?"
        logger.warn err.message
      end
    end
  end
```

The define_write_method method builds a *String of Code (141)* that is evaluated by class_eval. For example, if you call description=, then evaluate_attribute_method evaluates this String of Code:

```
def description=(new_value);write_attribute('description', new_value);end
```

Thus the description= method is born. A similar process happens for description, description?, and the accessors for all the other database columns.

Here's a recap of what we've covered so far. When you access an attribute for the first time, that attribute is a *Ghost Method (57)*. ActiveRecord::Base#method_missing takes this chance to turn the Ghost Method into a real method. While it's there, method_missing also dynamically defines read, write, and query accessors for all the other database columns. The next time you call that attribute or another database-backed attribute, you find a real accessor method waiting for you, and you don't have to enter method_missing anymore.

However, this logic doesn't apply to each and every attribute accessor, as you'll discover by looking at the second half of method_missing.

Attributes That Stay Dynamic

As it turns out, there are cases where Active Record doesn't want to define attribute accessors. For example, think of attributes that are not backed by a database column, such as calculated fields:

part2/ar_attribute_methods.rb

```
my_query = "tasks.*, (description like '%garage%') as heavy_job"
task = Task.find(:first, :select => my_query)
task.heavy_job?  # => true
```

Attributes like heavy_job can be different for each object, so there's no point in generating *Dynamic Methods (51)* to access them. The second half of method_missing deals with these attributes: